Spring Boot — Lecture 6

Circular Dependency, Bean Scopes & Lazy/Eager Initialization

1. What is Circular Dependency?

Circular dependency occurs when two (or more) classes depend on each other — Class A needs Class B to be created, and Class B needs Class A to be created. Neither can be instantiated first, so Spring gets stuck.

```
// Normal dependency chain — no problem
// PaymentGateway  <--  PaymentService  <--  OrderService
// Spring creates: PaymentGateway → PaymentService → OrderService (left to right)

// Circular dependency — PROBLEM
// OrderService needs PaymentService
// PaymentService needs OrderService
// Who gets created first? Spring cannot decide → exception!
```

Circular dependency with constructor injection:

```
@Component
public class OrderService {
    private PaymentService paymentService;

    public OrderService(PaymentService paymentService) {  // needs PaymentService
        this.paymentService = paymentService;
    }
}

@Component
public class PaymentService {
    private OrderService orderService;

    public PaymentService(OrderService orderService) {    // needs OrderService
        this.orderService = orderService;
    }
}
// Result: BeanCurrentlyInCreationException
// Spring error: "Is there an unresolvable circular reference?"
```

Key insight: circular dependency is NOT a Spring problem — it's a bad code design problem. You can reproduce the same infinite loop with plain Java using new keywords.

Plain Java circular dependency = StackOverflowError:

```
public class A {
    B b;
    public A() {
        this.b = new B();  // A creates B
        System.out.println("A created");
    }
}

public class B {
    A a;
    public B() {
        this.a = new A();  // B creates A → A creates B → B creates A → ∞
        System.out.println("B created");
    }
}

new A();  // → StackOverflowError
```

2. How Spring Detects Circular Dependency

With constructor injection, Spring detects the circular reference early — before even trying to instantiate. It tracks which beans are 'currently in creation' and throws BeanCurrentlyInCreationException when it sees a loop.

Step	What Spring does
1	Start creating OrderService bean
2	Constructor needs PaymentService — mark OrderService as 'in creation'
3	Start creating PaymentService bean
4	Constructor needs OrderService — but OrderService is already 'in creation'!
5	Throw BeanCurrentlyInCreationException — circular reference detected

3. Resolving Circular Dependency with Field/Setter Injection

Unlike constructor injection, field and setter injection separate object creation from dependency injection. Spring can create both empty objects first, then wire them together afterward.

How Spring resolves it internally:

Step	What Spring does (field/setter injection)
1	Create empty OrderService object (no dependencies yet)
2	Create empty PaymentService object (no dependencies yet)
3	Inject PaymentService into OrderService via field/@Autowired
4	Inject OrderService into PaymentService via field/@Autowired
5	Both beans fully wired — circular dependency resolved

Field injection — resolves circular dependency:

```
@Component
public class OrderService {
    @Autowired
    private PaymentService paymentService;  // field injection — no constructor needed

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}

@Component
public class PaymentService {
    @Autowired
    private OrderService orderService;  // field injection

    public void pay() {
        System.out.println("Payment done");
        orderService.getOrderDetails();  // calls back to OrderService
    }
}
// Result: Works! Spring creates both empty objects, then wires them.
```

This works in plain Spring Core. However, Spring Boot 2.6+ disables circular references by default. Set spring.main.allow-circular-references=true in application.properties to re-enable it (not recommended).

4. The Right Fix — Refactor, Don't Resolve

Resolving circular dependency with field injection just hides the real problem. The root cause is a violation of the Single Responsibility Principle — the two classes have entangled responsibilities.

Bad design (circular):

```
// OrderService.placeOrder() calls PaymentService.pay()
// PaymentService.pay() calls back OrderService.getOrderDetails()
//                                  ↑ This is wrong!
// PaymentService should NOT be calling back into OrderService.
```

Fixed design (linear dependency):

```
@Component
public class OrderService {
    private PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void placeOrder() {
        paymentService.pay();           // delegate payment to PaymentService
        getOrderDetails();              // then handle order details OURSELVES
        System.out.println("Order placed");
    }

    public void getOrderDetails() {
        System.out.println("Order details");
    }
}

@Component
public class PaymentService {
    // No dependency on OrderService at all!
    public void pay() {
        System.out.println("Payment done");
        // NOT calling back into OrderService — not its responsibility
    }
}
// Dependency is now one-directional: OrderService → PaymentService
```

The key question: WHY does PaymentService need to call back into OrderService? If it needs to fetch order details, that should be handled by the caller (OrderService). PaymentService's job is ONLY to process payments.

5. Side Note — @Configuration is also a @Component

The @Configuration annotation internally uses @Component. This means AppConfig itself is also a Spring-managed bean. Spring creates an AppConfig bean in the IoC Container, which is how it can call @Bean methods on it.

```
// Inside Spring source code — @Configuration meta-annotation:
@Component      // ← AppConfig IS a component!
public @interface Configuration { ... }

// This means Spring creates AppConfig bean in IoC Container,
// then calls @Bean methods ON that bean object to get other beans.
```

6. Bean Scopes — Singleton vs Prototype

Bean scope controls how many instances Spring creates for a given class and when they are created.

Scope	How many objects?	When created?	Use case
Singleton (default)	One per Bean Definition in the IoC Container	Eagerly — at container startup	Stateless classes: services, managers, repositories

Scope	How many objects?	When created?	Use case
Prototype	New object every time it is requested	Lazily — only when requested	Stateful classes: objects with unique per-user data
Request (web only)	One per HTTP request	When request arrives	Web: per-request data
Session (web only)	One per HTTP session	When session starts	Web: per-user session data
Application (web only)	One per ServletContext	At application start	Web: app-wide shared data

7. Singleton Scope — Default

By default, every Spring bean is singleton. One object is created per Bean Definition and the same object is returned every time it is requested.

```
@Component  // or @Scope("singleton") – same thing
public class OrderService {
    public OrderService() {
        System.out.println("OrderService created");
    }
}

// In Main:
OrderService order1 = context.getBean(OrderService.class);
OrderService order2 = context.getBean(OrderService.class);

System.out.println(order1 == order2);  // true – same object!
// Console: "OrderService created" printed ONCE
```

Singleton applies per Bean Definition, not per class:

```
// AppConfig.java
@Bean
public OrderService getOrder() {
    return new OrderService();   // Bean Definition 1
}

@Bean
public OrderService getOrder2() {
    return new OrderService();   // Bean Definition 2
}
// Result: TWO OrderService objects in the IoC Container
// Each is itself a singleton – but there are 2 bean definitions = 2 objects
// "OrderService created" prints TWICE
```

> *Important: Singleton in Spring is NOT the same as the Singleton Design Pattern. The design pattern enforces one object per JVM. Spring singleton means one object per Bean Definition — you can still use 'new' to create more outside Spring.*

8. Prototype Scope

With prototype scope, Spring creates a new object every time the bean is requested — whether via getBean() or as a dependency injection.

```
@Component
@Scope("prototype")
public class OrderService {
    public OrderService() {
        System.out.println("OrderService created");
    }
}
```

```
// In Main:
OrderService order1 = context.getBean(OrderService.class);
OrderService order2 = context.getBean(OrderService.class);

System.out.println(order1 == order2);  // false — different objects!
// Console: "OrderService created" printed TWICE

// Also when injected as dependency in A and B:
@Component class A { @Autowired OrderService os; }  // new object
@Component class B { @Autowired OrderService os; }  // another new object
// Console: "OrderService created" printed FOUR times total
```

Scenario	Singleton	Prototype
context.getBean() called twice	Same object returned both times	Two different objects created
Injected into classes A and B	Both A and B get the same object	A and B each get their own new object
Object creation timing	At container startup	Only when requested
Total objects in IoC Container	Exactly 1 per bean definition	1 new per request — not stored in container

9. When to Use Which Scope

Use Singleton when...	Use Prototype when...
Class is stateless — no per-instance data	Class is stateful — has unique per-instance data
Same logic needs to run for all callers	Each caller needs its own independent copy
Best for: Service classes, Manager classes, Repository classes	Best for: User objects, Order objects, DTO/model objects
Example: OrderService (processes orders but holds no order-specific state)	Example: User class with name, age — each user is different

```
// SINGLETON — correct for stateless service class
@Component  // default scope = singleton
public class OrderService {
    public void placeOrder() { /* business logic */ }
    // No instance fields holding state — safe as singleton
}

// PROTOTYPE — correct for stateful data class
@Component
@Scope("prototype")
public class User {
    private String name;  // each user has different name
    private int age;      // each user has different age
    // Must be prototype — sharing one User object makes no sense
}
```

10. Eager vs Lazy Initialization

Initialization refers to WHEN Spring creates the bean object.

Type	When created	Default for	Changeable?
Eager	At IoC container startup — before any code uses the bean	Singleton beans	Yes — add @Lazy to make it lazy

Type	When created	Default for	Changeable?
Lazy	Only when first requested — via getBean() or dependency injection	Prototype beans	Prototype is always lazy — cannot be made eager

Eager (default for singleton):

```
@Component  // singleton by default = eager by default
public class OrderService {
    public OrderService() {
        System.out.println("OrderService created");
    }
}

// In Main — even with NOTHING called on the bean:
ApplicationContext context =
    new AnnotationConfigApplicationContext(AppConfig.class);
// Console: "OrderService created" — created at startup, no request needed!
```

Lazy — using @Lazy on a singleton:

```
@Component
@Lazy  // override default eager behavior
public class OrderService {
    public OrderService() {
        System.out.println("OrderService created");
    }
}

// In Main:
ApplicationContext context =
    new AnnotationConfigApplicationContext(AppConfig.class);
// Console: (nothing yet — bean not created)

OrderService order = context.getBean(OrderService.class);
// Console: "OrderService created" — NOW it's created (when first requested)
```

Prototype is always lazy (cannot be changed):

```
@Component
@Scope("prototype")
public class OrderService {
    public OrderService() {
        System.out.println("OrderService created");
    }
}

ApplicationContext context =
    new AnnotationConfigApplicationContext(AppConfig.class);
// Console: (nothing) — prototype beans NEVER created at startup

OrderService o1 = context.getBean(OrderService.class);
// Console: "OrderService created" — created now

OrderService o2 = context.getBean(OrderService.class);
// Console: "OrderService created" again — NEW object every time
```

11. @Lazy on Dependency Injection Point

@Lazy can also be placed on an @Autowired field or constructor parameter to delay the injection of that specific dependency.

```
@Component
public class OrderService {
    @Autowired
```

```
    @Lazy  // inject a proxy here — real PaymentService created on first use
    private PaymentService paymentService;

    public OrderService() {
        System.out.println("OrderService created");
    }

    public void placeOrder() {
        paymentService.pay();  // PaymentService actually created HERE (first call)
        System.out.println("Order placed");
    }
}

// Startup output:
//   "OrderService created"     ← OrderService bean created (eager singleton)
//   (PaymentService NOT yet created — proxy inserted instead)

// After context.getBean(OrderService.class).placeOrder():
//   "PaymentService created"   ← Now it's created (on first use)
//   "Payment done"
//   "Order placed"
```

When @Lazy is on an injection point, Spring inserts a proxy object instead of the real bean. The real bean is only created when one of its methods is first called.

12. Global Lazy Initialization

You can make ALL beans lazy globally via application.properties:

```
# application.properties
spring.main.lazy-initialization=true   # all beans become lazy

# To make one specific bean eager while rest are lazy:
@Component
@Lazy(false)   # override global lazy — make this one eager
public class CriticalService { ... }
```

Setting	Effect
Default (nothing set)	Singleton beans are eager. Prototype beans are lazy.
@Lazy on a class	That singleton bean becomes lazy. Prototype unchanged (already lazy).
spring.main.lazy-initialization=true	ALL beans become lazy globally.
@Lazy(false) on a class	Only useful when global lazy is true — makes that specific bean eager again.
spring.main.allow-circular-references=true	Allows circular dependencies in Spring Boot 2.6+. Not recommended.

13. Using @Lazy to Resolve Circular Dependency

@Lazy on a constructor/field injection point can break a circular dependency by inserting a proxy. But again — this is not the recommended approach. Refactoring the design is always better.

```
@Component
public class OrderService {
    private PaymentService paymentService;

    public OrderService(@Lazy PaymentService paymentService) {
        // @Lazy inserts a proxy — PaymentService not created yet
        // OrderService can be fully created without PaymentService existing
        this.paymentService = paymentService;
    }
```

```
    public void placeOrder() {
        paymentService.pay();  // NOW PaymentService is actually created
        System.out.println("Order placed");
    }
}

@Component
public class PaymentService {
    private OrderService orderService;

    public PaymentService(OrderService orderService) {
        // OrderService already exists by the time this runs
        this.orderService = orderService;
    }
}
```

Prefer: refactor so the circular dependency does not exist in the first place. Ask: why does PaymentService need OrderService? Usually the caller (OrderService) should handle that responsibility itself.

14. Full Summary — Scope + Initialization

Combination	Objects created	When	Can change?
Singleton + Eager (default)	1 per Bean Definition	At container startup	Yes — add @Lazy
Singleton + Lazy (@Lazy)	1 per Bean Definition	On first request	Yes — remove @Lazy
Prototype (always lazy)	New object every request	On every request	Cannot be made eager
Global lazy (spring.main.lazy-initialization=true)	All beans lazy	On first request each	Override per-class with @Lazy(false)

Why Spring defaults to eager initialization:

- Fail Fast — all wiring errors (missing beans, ambiguous beans, circular deps) are caught at startup.
- If lazy, bugs only surface when that code path is hit — possibly in production.
- Spring itself recommends eager as the safer default for most applications.

Next Lecture: Bean Lifecycle — @PostConstruct, @PreDestroy, InitializingBean, DisposableBean